SOUP INTERNSHIP · TAKE-HOME TASK

Backend Engineer

Work: 4–6 h · Deadline: 48 h · Any stack · Run: docker compose up

TASK

A training-job queue service with several workers, and an answer to one question: will it survive production? Green happy-path tests prove nothing.

REQUIREMENTS

Create a job (with an idempotency key), get status, list with pagination, cancel. States: queued → running → succeeded / failed / cancelled. Storage: Postgres or SQLite, explain your choice. At least 4 workers. Emulate "execution" with a side effect (a row or file write) so you can count how many times a job really ran.

1. Design before code. One page: how a worker claims a job, how you detect a dead worker, what happens when a running job is cancelled, which guarantee you give (at-least-once, at-most-once or exactly-once effect) and why.

2. Prove it with a chaos test. Write a harness: 10,000 jobs, 8 workers, a random worker killed with kill -9 about every 2 seconds, while repeated POSTs with the same keys and cancellations run in parallel. Check the invariants: no job is lost, none has more than one side effect, none is stuck in running, final states are consistent. Include the raw output and the number of runs. What does the harness NOT catch?

3. Silent failures. What can break without errors: a double claim, a "zombie" worker that came back after its lease expired and wrote a stale result, a cancel/complete race, a duplicate on request retry, transaction isolation level, clock skew. Show how you checked each.

4. Verdict. SHIP or DON'T SHIP at 100 requests per second. If DON'T SHIP, what to fix first.

DELIVERABLES

Repository · design note · the harness and raw output of all runs, including failed ones · one paragraph on what surprised you and what still concerns you. Submit by email reply to team@trysoup.dev. Don't edit or hide failed results: a clear negative beats a quiet success. If something breaks, tell us what happened, what you tried, and what you learned.

AI TOOLS

Allowed (we use them too). Note where you used them, which suggestions you accepted, and what you verified, changed, or redid yourself. Not a penalty: we want to see how you work and where you spot an unreliable tool.

LIVE DISCUSSION

40 minutes, after submission. Don't pre-write answers; we want to think it through with you. Possible questions: A worker froze for 5 minutes and came back: what happens? Where is your design weakest? What breaks first at 10x load? How do you know 10,000 jobs was enough?

SCORING

Correctness under failure — 30% · silent failures — 20% · design and code quality — 15% · live discussion — 35%. We're not just checking whether a metric improved. We want an engineer who questions assumptions, checks evidence, and understands what the system is actually doing.

Good luck. Be precise, honest, and curious.